

Opis API-ja mikroservisa

1. Arhitektura sistema

Sistem se sastoji od tri mikroservisa koji pristupaju zajedничој PostgreSQL bazi podataka. Svaki servis implementira iste operacije (upis, citanje, selektivno citanje, agregacije) ali koristi razlicit komunikacioni protokol i format serijalizacije.

Servis	Tehnologija	Protokol	Port	Format podataka
REST	ASP.NET Core (C#)	HTTP/1.1	5000	JSON
gRPC	Python (grpcio)	HTTP/2	50051	Protobuf (binarno)
GraphQL	ASP.NET Core + Hot Chocolate (C#)	HTTP/1.1	5002	JSON

Konteјnerizacija je realizovana pomocu Docker Compose-a. PostgreSQL baza ima healthcheck, a svi servisi zavise od nje (`depends_on: condition: service_healthy`).

2. Baza podataka

PostgreSQL 16 sa jednom tabelom:

```
CREATE TABLE sensor_readings (  
  id          BIGSERIAL PRIMARY KEY,  
  ts          TIMESTAMPTZ NOT NULL,  
  device_id   VARCHAR(20) NOT NULL,  
  co          DOUBLE PRECISION,  
  humidity    DOUBLE PRECISION,  
  light       BOOLEAN,  
  lpg         DOUBLE PRECISION,  
  motion      BOOLEAN,  
  smoke       DOUBLE PRECISION,  
  temp        DOUBLE PRECISION  
);
```

Indeksi za optimizaciju upita:

- `idx_readings_device_id` — pretraga po uredjaju
- `idx_readings_ts` — pretraga po vremenu
- `idx_readings_device_ts` — kompozitni indeks za filtriranje po uredjaju i vremenu

Dataset sadrzi 405,184 ocitavanja sa 3 IoT uredjaja (Smart Home senzori), vremenski opseg jul 2020. Podaci su uvezeni iz CSV fajla kroz staging tabelu sa konverzijama tipova (UNIX epoch u TIMESTAMPTZ, tekst u boolean).

PostgreSQL je konfigurisan sa `max_connections=500` da podrzi testove sa velikim brojem konkurentnih konekcija.

3. REST servis (ASP.NET Core)

Klasican REST API sa JSON serijalizacijom. Dokumentovan kroz Swagger/OpenAPI specifikaciju, dostupnu na <http://localhost:5000/swagger>.

Koristi Entity Framework Core sa Npgsql provajderom za pristup bazi. Connection pool je povecan na 200 konekcija za podrsku veceg opterecenja.

Endpointi

POST `/api/readings` — Upis jednog ocitavanja

Prima JSON objekat sa senzorskim podacima i upisuje ga u bazu.

Request body:

```
{
  "ts": 1594512000,
  "deviceId": "b8:27:eb:bf:9d:51",
  "co": 0.005,
  "humidity": 51.0,
  "light": false,
  "lpg": 0.007,
  "motion": false,
  "smoke": 0.02,
  "temp": 22.7
}
```

Odgovor: 201 Created sa kreiranim objektom.

POST /api/readings/batch — Batch upis

Prima niz očitavanja i upisuje ih odjednom.

Request body:

```
{
  "readings": [
    { "ts": 1594512000, "deviceId": "...", "co": 0.005, ... },
    { "ts": 1594512001, "deviceId": "...", "co": 0.006, ... }
  ]
}
```

Odgovor: 200 OK sa brojem upisanih redova.

GET /api/readings — Citanje po uređaju i vremenu

Parametri (query string):

- **deviceId** (obavezno) — MAC adresa uređaja
- **from** (opciono) — pocetak vremenskog opsega (ISO 8601)
- **to** (opciono) — kraj vremenskog opsega (ISO 8601)
- **limit** (opciono, podrazumevano 100) — maksimalan broj rezultata

Primer:

```
GET /api/readings?deviceId=b8:27:eb:bf:9d:51&from=2020-07-12T00:00:00Z&to=2020-07-12T01:00:00Z&limit=5
```

Odgovor: JSON niz sa svim poljima svakog očitavanja. Ovo je primer over-fetching problema — klijent dobija sva polja cak i ako mu trebaju samo neka.

GET /api/readings/select — Selektivno citanje

Isti parametri kao gore, plus:

- **fields** (obavezno) — lista polja odvojena zarezima (npr. temp,humidity)

Primer:

```
GET /api/readings/select?deviceId=b8:27:eb:bf:9d:51&fields=temp,humidity&from=2020-07-12T00:00:00Z&to=2020-07-12T01:00:00Z&limit=5
```

Odgovor: JSON niz sa samo trazanim poljima (plus ts i deviceId). Ovo je pokusaj resavanja over-fetching problema na serverskoj strani, ali zahteva poseban endpoint.

GET /api/readings/aggregate — Agregacije

Parametri:

- **deviceId** (obavezno)
- **from** (obavezno)
- **to** (obavezno)

Primer:

```
GET /api/readings/aggregate?deviceId=b8:27:eb:bf:9d:51&from=2020-07-12T00:00:00Z&to=2020-07-13T00:00:00Z
```

Odgovor:

```
{
  "deviceId": "b8:27:eb:bf:9d:51",
  "from": "2020-07-12T00:00:00Z",
  "to": "2020-07-13T00:00:00Z",
  "count": 135169,
  "avgCo": 0.0045,
  "minCo": 0.0028,
  "maxCo": 0.0076,
  "avgHumidity": 76.01,
  "minHumidity": 49.0,
  "maxHumidity": 100.0,
  "avgTemp": 23.12,
  "minTemp": 19.0,
  "maxTemp": 29.5,
  ...
}
```

4. gRPC servis (Python)

Implementiran u Pythonu sa `grpcio` bibliotekom. Koristi Protobuf za definiciju interfejsa i binarnu serijalizaciju podataka. Pristupa bazi direktno preko `psycopg2` sa `ThreadPool`-om.

Interfejs je definisan u `.proto` fajlu koji opisuje servis, RPC metode i strukture poruka. Iz `.proto` fajla se generise Python kod za server i klijent.

Proto definicija

```
service SensorService {
  rpc IngestReading (SensorReading) returns (IngestResponse);
  rpc IngestBatch (BatchRequest) returns (IngestResponse);
  rpc GetReadings (ReadingsRequest) returns (ReadingsResponse);
  rpc GetSelectedFields (SelectedFieldsRequest) returns (SelectedFieldsResponse);
  rpc GetAggregation (AggregationRequest) returns (AggregationResponse);
}
```

Poruke (message tipovi)

SensorReading — osnovna struktura jednog očitavanja:

```
message SensorReading {
  double ts = 1;
  string device_id = 2;
  double co = 3;
  double humidity = 4;
  bool light = 5;
  double lpg = 6;
  bool motion = 7;
  double smoke = 8;
  double temp = 9;
}
```

Svako polje ima redni broj (1-9) umesto tekstualnog imena. To je razlog zasto je Protobuf manji od JSON-a — na zici se prenose samo brojevi i vrednosti, bez imena polja.

ReadingsRequest — parametri za citanje:

```
message ReadingsRequest {
  string device_id = 1;
  double from_ts = 2;
  double to_ts = 3;
  int32 limit = 4;
}
```

SelectedFieldsRequest — za selektivno citanje, sadrzi listu trazenih polja:

```
message SelectedFieldsRequest {
  string device_id = 1;
  double from_ts = 2;
  double to_ts = 3;
  repeated string fields = 4;
  int32 limit = 5;
}
```

AggregationResponse — rezultat agregacije sa avg/min/max za svaku numericku kolonu:

```
message AggregationResponse {
  double avg_co = 1;
  double min_co = 2;
  double max_co = 3;
  double avg_humidity = 4;
  double min_humidity = 5;
  double max_humidity = 6;
  double avg_temp = 13;
  double min_temp = 14;
  double max_temp = 15;
  int64 count = 16;
  ...
}
```

RPC metode

Metoda	Ulaz	Izlaz	Opis
IngestReading	SensorReading	IngestResponse	Upis jednog očitavanja
IngestBatch	BatchRequest	IngestResponse	Batch upis
GetReadings	ReadingsRequest	ReadingsResponse	Citanje po uređaju i vremenu
GetSelectedFields	SelectedFieldsRequest	SelectedFieldsResponse	Selektivno citanje (konkretna polja)
GetAggregation	AggregationRequest	AggregationResponse	Agregacije nad opsegom podataka

Testiranje

gRPC nema graficki interfejs kao Swagger. Za rucno testiranje koristi se `grpcurl` alat:

```
grpcurl -plaintext localhost:50051 list
grpcurl -plaintext -d '{"device_id":"b8:27:eb:bf:9d:51","from_ts":1594512000,"to_ts":1594515600,"limit":3}' localhost:50051 sensor.S
```

Server ima ukljucenu reflection koja omogucava `grpcurl`-u da dinamicki otkriva dostupne servise i metode.

5. GraphQL servis (ASP.NET Core + Hot Chocolate)

Implementiran u ASP.NET Core sa Hot Chocolate bibliotekom za GraphQL. Koristi `IDbContextFactory` za kreiranje `DbContext` instanci po zahtevu (umesto singleton-a), sto omogucava paralelnu obradu upita.

Graficki interfejs za testiranje (Banana Cake Pop) dostupan na <http://localhost:5002/graphql>.

Schema

GraphQL ima jednu ulaznu tacku (`/graphql`) za sve operacije. Klijent u samom upitu definise koja polja zeli da dobije — to je osnovna razlika u odnosu na REST gde server odredjuje strukturu odgovora.

Queries (citanje)

readings — dohvata očitavanja za uređaj u vremenskom opsegu:

```

query {
  readings(
    deviceId: "b8:27:eb:bf:9d:51"
    from: "2020-07-12T00:00:00Z"
    to: "2020-07-12T01:00:00Z"
    limit: 5
  ) {
    temp
    humidity
  }
}

```

Klijent navodi samo `temp` i `humidity` i dobija samo ta dva polja u odgovoru. Ako doda jos polja (npr. `co`, `smoke`), odgovor ce sadrzati i njih. Ovo resava over-fetching problem bez potrebe za posebnim endpointom kao kod REST-a.

Primer sa svim poljima:

```

query {
  readings(deviceId: "b8:27:eb:bf:9d:51", from: "2020-07-12T00:00:00Z", to: "2020-07-13T00:00:00Z", limit: 3) {
    ts deviceId co humidity light lpg motion smoke temp
  }
}

```

aggregation — agregacije nad opsegom podataka:

```

query {
  aggregation(
    deviceId: "b8:27:eb:bf:9d:51"
    from: "2020-07-12T00:00:00Z"
    to: "2020-07-13T00:00:00Z"
  ) {
    count avgTemp minTemp maxTemp avgHumidity minHumidity maxHumidity
  }
}

```

I ovde klijent bira koja polja agregacije zeli. Ako mu treba samo `avgTemp`, moze da trazi samo to.

Mutations (upis)

ingestReading — upis jednog očitavanja:

```

mutation {
  ingestReading(input: {
    ts: 1594512000
    deviceId: "test:device:01"
    co: 0.005
    humidity: 55.0
    light: false
    lpg: 0.007
    motion: true
    smoke: 0.02
    temp: 23.5
  }) {
    id ts temp
  }
}

```

Odgovor mutacije takodje podleze selekciji polja — klijent bira sta zeli da vidi kao potvrdu upisa.

ingestBatch — batch upis:

```
mutation {
  ingestBatch(inputs: [
    { ts: 1594512000, deviceId: "test:01", co: 0.005, humidity: 55.0, light: false, lpg: 0.007, motion: true, smoke: 0.02, temp: 20.0 },
    { ts: 1594512001, deviceId: "test:01", co: 0.006, humidity: 56.0, light: true, lpg: 0.008, motion: false, smoke: 0.03, temp: 21.0 }
  ]) {
    success count
  }
}
```

Tipovi

Tip	Polja	Opis
SensorReading	id, ts, deviceId, co, humidity, light, lpg, motion, smoke, temp	Jedno senzorsko očitavanje
AggregationResult	count, avgCo, minCo, maxCo, avgHumidity, minHumidity, maxHumidity, avgLpg, minLpg, maxLpg, avgSmoke, minSmoke, maxSmoke, avgTemp, minTemp, maxTemp	Rezultat agregacije
ReadingInput	ts, deviceId, co, humidity, light, lpg, motion, smoke, temp	Ulazni tip za mutacije
IngestResult	success, count	Rezultat batch upisa

6. Poredjenje pristupa

Definisanje interfejsa

Aspekt	REST	gRPC	GraphQL
Definicija	Implicitna (kontroleri + Swagger)	Eksplcitna (.proto fajl)	Eksplcitna (schema)
Dokumentacija	Swagger UI (automatski generisana)	Potreban grpcurl ili protoset	Banana Cake Pop (ugradjeni)
Generisanje koda	Nije potrebno	Da (protoc kompajler)	Nije potrebno
Tipizacija	Slaba (JSON nema tipove)	Stroga (Protobuf tipovi)	Stroga (GraphQL tipovi)

Fleksibilnost upita

Aspekt	REST	gRPC	GraphQL
Selekcija polja	Zahteva poseban endpoint (/select)	Zahteva posebnu RPC metodu	Ugradjeno — klijent bira polja u upitu
Novi endpoint/metoda za novi use-case	Da	Da (novi RPC + poruke u .proto)	Ne — isti endpoint, drugi upit
Over-fetching	Da, osim ako ne napravimo /select	Delimicno (zavisi od definicije poruka)	Ne — klijent trazi samo sto mu treba

Serijalizacija

Aspekt	REST (JSON)	gRPC (Protobuf)	GraphQL (JSON)
Format	Tekstualni	Binarni	Tekstualni
Imena polja u odgovoru	Da	Ne (redni brojevi)	Da
Citljivost	Ljudski citljiv	Nije bez .proto fajla	Ljudski citljiv
Velicina	Najveca	Najmanja (2-4x manja)	Srednja (manja ako se biraju polja)